

An Empirical Study on the Developers' Perception of Software Coupling

Gabriele Bavota¹, Bogdan Dit², Rocco Oliveto³, Massimiliano Di Penta⁴, Denys Poshyvanyk², Andrea De Lucia¹

¹University of Salerno, Fisciano (SA), Italy

²The College of William and Mary, Williamsburg, VA, USA

³University of Molise, Pesche (IS), Italy

⁴University of Sannio, Benevento, Italy

Abstract—Coupling is a fundamental property of software systems, and numerous coupling measures have been proposed to support various development and maintenance activities. However, little is known about how developers actually perceive coupling, what mechanisms constitute coupling, and if existing measures align with this perception. In this paper we bridge this gap, by empirically investigating how class coupling—as captured by structural, dynamic, semantic, and logical coupling measures—aligns with developers' perception of coupling. The study has been conducted on three Java open-source systems—namely ArgoUML, JHotDraw and jEdit—and involved 64 students, academics, and industrial practitioners from around the world, as well as 12 active developers of these three systems. We asked participants to assess the coupling between the given pairs of classes and provide their ratings and some rationale. The results indicate that the peculiarity of the semantic coupling measure allows it to better estimate the mental model of developers than the other coupling measures. This is because, in several cases, the interactions between classes are encapsulated in the source code vocabulary, and cannot be easily derived by only looking at structural relationships, such as method calls.

Index Terms—Software Coupling, Empirical Studies.

I. INTRODUCTION

Coupling has been defined by Stevens *et al.* [1] as “the measure of the strength of association established by a connection from one module to another”. It is considered as one of the fundamental principles of software engineering, with a strong influence on comprehension and maintenance of large software systems. Understanding the coupling between entities is useful for a variety of software development or maintenance activities, such as assessing software quality [2], [3], predicting fault-proneness [4], [5], [6], supporting change impact analysis [7], [8], [9], [10], re-engineering [11], reuse [12], change propagation [13], and clone management [14].

The importance of coupling has motivated researchers to define different types of measures that capture various aspects of software quality. Most of the coupling measures are *structural* and they statically analyze the source code to capture different relations between entities (e.g., classes), such as the number of calls between two entities, variable accesses, or inheritance relations. Over the years, several alternative measures of coupling have been proposed, such as *dynamic* coupling, which takes into account call relationships occurring during program execution [15], *semantic* coupling, which exploits relations captured from the source code lexicon using Infor-

mation Retrieval techniques [8], or *logical* coupling, which uses historical data to identify co-changing artifacts [16]. The rationale behind these three coupling measures—dynamic, semantic, and logical—is to capture relations between software artifacts, which are not captured by structural coupling.

Some empirical studies evaluated the potential use of many existing coupling measures and how these measures could be used in a complementary manner [17], [8]. However, despite these attempts to characterize different coupling measures, it is still unclear to what extent such coupling measures reflect developers' perception of coupling between classes, and what sources of information align better with their perception. In essence, the following issue still remains completely unaddressed in the literature after several decades of research:

To what extent does the coupling level captured by measures reflects the strength of coupling perceived by developers?

To answer this question, we conducted an empirical study aimed at investigating how different kinds of coupling measures reflect developers' perception of coupling. The study has been conducted on three open source systems, namely ArgoUML¹, JHotDraw², and jEdit³, and involved 64 students, academics and practitioners, as well as 12 professional software developers that have been active contributors to these open-source systems. Firstly, we analyzed the extent to which these coupling measures overlap. Secondly, we selected a number of class pairs from each system—having both high and low coupling according to the studied measures—and asked participants to assess the strength of those couplings. Thirdly, we analyzed if the perceived coupling aligned with the one captured by the structural, dynamic, semantic, and logical coupling. In order to enrich the generalizability of the results achieved and gain a better understanding of these results, we also analyzed to what extent the experimented measures reflect the mental model of the developers grouping together (i.e., in the same package) classes that were originally grouped by the original developers. Specifically, we used these coupling measures to reconstruct the original modularization of the software systems using a state-of-the-art tool, Bunch [18], and

¹<http://argouml.tigris.org/>

²<http://www.jhotdraw.org/>

³<http://www.jedit.org/>

using the **Move Join eEffectiveness Measure** (MoJoFM) [19] to evaluate the resulting decomposition.

The contribution of our paper is threefold. First, our study and the results provide important insights into better understanding how developers perceive coupling among classes and which measures align with programmers' opinions. It is an important step in understanding the practical implications of software measurement theory and identifying important future directions. Second, we provide concrete results on how different coupling measures can be used for software modularization, and which measures agree with the original software designers. Finally, the study design, the materials, and the data are made publicly available in our online appendix⁴ for replication purposes or to support future studies aiming at evaluating other coupling measures. We believe that the results of this work are a stepping stone for current and future approaches on coupling measures, as well as new studies of software metrics that put developers in the loop.

Structure of the paper. Section II discusses the related literature. Sections III and IV describe the design of our empirical study and present the analysis of the achieved results, respectively. Section V discusses the threats to validity that could affected our study, while Section VI concludes the paper and presents the agenda for future work.

II. RELATED WORK

There are many existing coupling metrics that employ different types of information such as structural, dynamic, semantic, or logical. Most of these metrics define coupling at the class-level granularity by determining the degree to which two classes in an object-oriented system depend on one another. On the one hand, the majority of existing coupling metrics capture coupling between classes structurally. Coupling Between Objects (CBO), Response for a Class (RFC) [20] and Information flow-based coupling (ICP) [21] are among the representative structural coupling metrics. Due to space limitations we refer the interested reader to the unified framework for coupling measurement by Briand *et al.* [17], which provides an overview of the structural metrics.

Furthermore, semantic metrics define a coupling for classes based on textual information extracted from source code identifiers and comments [22]. One such semantic metric, CCBC (Conceptual Coupling Between Classes) [22], captures a new dimension of coupling not addressed by structural or dynamic measures. Logical (or evolutionary) coupling [23], [24], [25] utilizes information from repositories to capture the strength of relations among the artifacts (including source code) that are frequently co-changed. Such logical coupling metrics have been used for impact analysis [26], much like structural coupling metrics. Finally, Arisholm *et al.* [15] introduced dynamic import (i.e., object calls) and export (i.e., called by) metrics to capture the coupling between classes and objects at runtime.

While there have been a number of research papers defining and using coupling metrics for various tasks, there has been only one study [10] that investigated if (any) coupling metrics align with developers' opinions. However, the study by Revelle *et al.* [10] defined and evaluated only structural and semantic coupling metrics. Moreover, those metrics were defined at feature level only. Our study aims at bridging this gap and studying which metrics (and sources of information) are useful from a software developer's perspective. Beck and Diehl [27] measured the congruence between different types of coupling (e.g., structural, semantic, evolutionary, fan-out similarities, code clones and code ownership) and the modularity of a system. Their study on 16 Java systems revealed that (i) low coupling and high cohesion and (ii) information hiding are the two of the modularity principles that are used in practice. Our study complements Beck and Diehl's study, by identifying which types of coupling are perceived as more useful by developers. Moreover, in [28], [29] they showed that evolutionary coupling can be successfully used when performing software clustering if substantially evolutionary data is available.

The study by Counsell *et al.* [30] approached the issue of investigating the developers' perception of cohesion, similarly to what we have done for coupling, i.e., by asking 24 experienced and novice IT professionals to evaluate the cohesion of 10 classes from a Java system. They found that the perceived cohesion (i) is not correlated with the class size, (ii) is correlated to comments density, and (iii) does not depend on the developers' experience. The correlation they found with comments somehow reflect the findings of our paper concerning the capability of coupling to capture developers' perception.

III. EMPIRICAL STUDY DEFINITION AND DESIGN

The *goal* of this study is to analyze different types of coupling measures— structural, dynamic, semantic, and logical coupling— and investigate to what extent these measures reflect developers' perception of coupling.

The *context* consists of (i) *objects*, i.e., source code, execution traces, and change history of three Java open-source systems, namely ArgoUML, JHotDraw, and jEdit; and (ii) *subjects* providing their perception of the strength of coupling between entities, identified by the different coupling measures. We recruited both original developers working on these projects, as well as external developers: students, faculty members, and industrial programmers. As for the subject software systems, ArgoUML is an open-source UML modeling tool with advanced features, such as reverse engineering and code generation. JHotDraw is a Java framework for drawing 2D graphics, initially developed to illustrate the applicability of object-oriented design patterns. jEdit is an open-source textual editor developed in Java and supports source code editing. The choice of these three systems is motivated by the need to have: (i) systems of different sizes that are not too small nor too large, to allow developers to assess coupling among classes in the context of an entire system; (ii) systems belonging

⁴<http://www.distat.unimol.it/reports/coupling>

TABLE I
CHARACTERISTICS OF THE COUPLING MEASURES OF THE THREE OBJECT
SYSTEMS AND ANALYZED DATA

Measure	Characteristic	ArgoUML 0.34	JHotDraw 6.0b1	jEdit 2.4.1
Structural	KLOC	280	29	28
	# of classes	1,889	289	245
	Exploited metric	Information-flow-based Coupling (ICP)		
Dynamic	Stmt. coverage reached	66%	67%	86%
	Exploited metric	Import Coupling Class Dynamic Message (IC_CD)		
Semantic	Vocabulary Size	7,961	2,834	2,418
	Exploited metric	Conceptual Coupling Between Classes (CCBC)		
Logical	Period of analysis	1998-2006	2000-2005	2000-2012
	Exploited metric	Association rule-based Change Coupling		

to different problem domains, (iii) availability of sufficient historical data and (iv) the possibility to collect execution traces by manually invoking specific features, which capture the functionality that is exposed to the user through GUI. In addition, in this way we avoid the need to have an accurate test suite to derive execution scenarios. Table I presents the characteristics of three systems involved in our study, relevant for the four sources of information we used to obtain the coupling measures.

A. Research Questions

The study answers the following research questions:

- *RQ₁: To what extent are the sources of information for coupling measures complementary?* This research question is preliminary to the other two, and aims at determining whether the four coupling measures capture the same relations or whether, instead, there are links captured by some measures and not by others.
- *RQ₂: To what extent are the sources of information for coupling measures useful for capturing the strength of coupling as perceived by developers?* This research question is the core of our study, and aims at investigating how developers of these three projects and external ones, perceive the strength of coupling among classes as identified by our four measures.
- *RQ₃: To what extent do coupling measures reflect the mental model of developers when grouping together classes?* Finally, this research question considers the original system modularization as an “oracle” for assessing the coupling measures and determines to what extent a state-of-the-art modularization tool, Bunch [18], is able to produce—using the four measures—class clusters similar to the original ones.

B. Variables and Procedure

Independent Variables. The independent variables of our study are four representative and commonly used coupling measures, each one exploiting a different source of information. The first measure, the *structural coupling* is quantified through the information-flow-based coupling (ICP) [21], by statically analyzing the software source code⁵. ICP measures

the amount of information flowing into and out of a class via parameters through method invocation (i.e., the measure sums the number of parameters passed at each method invocation). Like the majority of coupling measures in literature, this measure is defined at the system level (i.e., for a given class c all method calls between c and *all* other classes in the system are taken into account). For our study we need to redefine ICP to take into account the coupling between a pair of classes. We used the pairwise ICP metric as redefined in [8], i.e., the ICP between a pair of classes c_i and c_j is measured as the number of method invocations in the class c_i to methods in the class c_j , weighted by the number of parameters of the invoked methods. Then, the overall information-flow-based coupling between the classes c_i and c_j is computed as the maximum between $ICP(c_i, c_j)$ and $ICP(c_j, c_i)$.

We collected execution traces by running the software on an instrumented Java Virtual Machine⁶. For each software system we collected a set of execution traces $T = \{t_1, t_2, \dots, t_N\}$. Each trace t_i was collected by manually exercising a subset of features of the system. For example, one feature associated with t_i was opening, saving, exporting and closing a file, or creating a UML diagram (for ArgoUML), drawing a shape (for JHotDraw) or editing some text (for jEdit). The dynamic coupling between two classes c_i and c_j was computed by slightly adapting the formula **Import Coupling Class Dynamic Message (IC_CD)** introduced by Arisholm *et al.* [15]. In its original form, the $IC_CD(c)$ metric counts the total number of distinct messages (i.e., method calls) sent from class c to all the other classes in the system. However, similarly to the ICP metric, for our purpose, we adapted the metric to quantify the dynamic coupling between pairs of classes. In other words, the adapted metric $IC_CD(c_i, c_j)$ counts all the distinct method calls from objects of type c_i to objects of type c_j . Due to space limitations, we refer the reader to the original formula defined by Arisholm *et al.* [15] in Table 2. The only difference is that our formula replaces $IC_CD(c_1)$ with $IC_CD(c_1, c_2)$.

The third measure, the *semantic coupling*, is represented by the **Conceptual Coupling Between Classes (CCBC)** metric [8]. The CCBC is based on the lexical information derived from comments and identifiers. Two classes are semantically related if the terms present in their comments and identifiers are similar. The conjecture is that if two classes contain similar terms, it is likely that developers used them to describe similar responsibilities. The CCBC between two classes c_i and c_j is computed as the average textual similarity between all unordered pairs of methods from class c_i and class c_j . The textual similarity is computed using Latent Semantic Indexing (LSI) [31]. The CCBC values are in $[0 \dots 1]$, where 0 represents two classes having a totally different lexicon, while 1 represents two classes containing exactly the same text.

The fourth and last measure is the *logical coupling*. To compute it we first extract change sets from the version control system. For ArgoUML and jEdit we rely on SVN change sets. JHotDraw uses CVS, therefore, we grouped together file

⁵Static analysis is performed by using Eclipse’s AST parser.

⁶We used Eclipse’s TPTP (<http://www.eclipse.org/tptp/>)

changes if the commit note and the committer name matched, and if the time interval between them was less than 200 seconds [32]. Then, we applied association rule discovery, similarly to the work by Ying *et al.* [24] and by Zimmermann *et al.* [32] (using the *R* package *arule* and, specifically, the Apriori [33] algorithm) to detect frequent itemsets in co-changes (i.e., groups of files frequently changing together). The rules are ranked in terms of (i) *support* of each itemset X , $\text{supp}(X)$ (i.e., the proportion of change sets that contain the learned itemset), and (ii) *confidence* of a rule $X \rightarrow Y$, defined as $\text{supp}(X \rightarrow Y) / \text{supp}(X)$ (i.e., the fraction of change sets containing X where Y also appears). We set the *confidence* threshold to 0.8 and the *support* threshold to 0.02. The rationale was that (i) the learned rules must be precise enough (hence, the high confidence threshold) and (ii) the itemsets may appear in a limited percentage of the change sets (hence, the low support threshold). Using these thresholds, we assumed that there is a logical coupling between two files if they appeared in a frequent itemset. Finally, we mapped files to classes by analyzing each file's source code.

Dependent Variables. To address these three research questions, we relied on different dependent variables collected through the procedures described below.

For **RQ₁**, given a pair of measures M_i and M_j , we compared the overlap between coupling links (i.e., pairs of coupled classes) by computing the intersection of the sets of coupling links identified by the two metrics. Note that in this research question we were not interested in analyzing the “value” of the identified coupling links (true or false), but only in estimating the complementarity of these four measures.

For **RQ₂**, we asked software developers to look into pairs of classes and determine the extent to which they were coupled. We selected a sample of 16 class pairs for each subject system adhering to the following process:

- 1) we computed the coupling metrics (structural, dynamic, semantic, and logical) between all pairs of classes in the system;
- 2) for each coupling measure M_i , we selected (i) two pairs of classes having the highest coupling (as stated by M_i) and (ii) two pairs of classes having the lowest coupling (as stated by M_i).

Thus, for each of the four coupling measures we selected four pairs of classes from each system (hence the total of 16 pairs of classes per system). Using this sample we wanted to investigate if the developers perception of high (low) coupling aligns with the values of our underlying software metrics.

We used the following procedure to recruit participants for our study:

- 1) *Developers working on the three open-source systems.* We sent an invitation only to the topmost committers for all the investigated classes⁷. In total, we invited 19 developers from ArgoUML, 7 from JHotDraw, and 15 from jEdit. We received responses from 6, 3, and 3

developers, respectively. In the following, we will refer to them as *original developers*.

- 2) *Undergraduate and graduate students, academics, and practitioners from around the world.* Note that while the original developers of the three systems were invited to evaluate only the pairs of classes extracted from the system that they were working on, academics and practitioners were invited to evaluate pairs of classes on all the three systems. Of the 96 invited, 64 (3 undergraduate students, 33 MS students, 16 PhD students, 7 faculty, and 5 people with industrial experience) participated in our study, evaluating the strength of coupling between the pairs of classes from all the systems. In the following discussion, we will refer to them as *external developers*.

Each developer received an email (available in our replication package) with instructions on how to perform the task and a link to the website where each developer could log in to visualize and rate the class pairs. More specifically, for each system, each of the 16 webpages provided to the developers contained (i) a class pair to analyze (class names only), (ii) hyper-links to the source code of these classes, (iii) a hyper-link to the source code of the complete project, (iv) an input form containing five radio buttons representing a Likert scale [34] ranging from 1 (two classes are *not coupled*) to 5 (two classes are *strongly coupled*), and (v) a text field where the participant could add explanations for their assessment. Developers were allowed up to four weeks to complete this survey. The study was designed to last no more than 2 hours.

The results of **RQ₂** were analyzed through box-plots and statistical analysis using the Mann-Whitney test [35] (the test was performed only on data collected from the external developers, due to the limited number of original developers). We grouped the pairs of classes evaluated by the developers into eight different groups: pairs having $\{low, high\} \times \{semantic, structural, logical, dynamic\}$ coupling. Then, considering two groups at a time (e.g., *high semantic coupling* vs. *high structural coupling*), we used the Mann-Whitney test to analyze the statistical significance of the difference between the coupling perceived by the developers in pairs of classes classified as *high* or *low* coupled by the two different coupling measures. The results were intended as statistically significant at $\alpha = 0.05$. However, since we performed multiple tests, we adjusted our p-values using the Holm's correction procedure [36]. This procedure sorts the p-values resulting from n tests in ascending order, multiplying the smallest by n , the next by $n - 1$, and so on. We also estimated the magnitude of the difference between the employed metrics. We used Cliff's Delta (or d), a non-parametric effect size measure [37] for ordinal data. The effect size is small for $d < 0.33$ (positive as well as negative values), medium for $0.33 \leq d < 0.474$ and large for $d \geq 0.474$ [37]. Clearly, our statistical analysis was applicable only to the data provided by external developers, since the number of data points obtained from the original developers was too few to run statistical tests. However, we used the data from the original developers to corroborate the

⁷To identify such developers, we analyzed the history of software changes.

to ArgoUML, the *semantic* measure is the one that better approximates (for highly coupled classes) the coupling perceived by both original and external developers (median=5 for original developers, and median=4 for the external ones). Indeed, the Mann-Whitney test reports a statistically significant difference between the coupling perceived by external developers on pairs of classes having high *semantic* coupling and the pairs of classes reported as highly coupled by the other three measures (p-value always < 0.01, medium/high effect size). As for the other three measures, the *structural* measure is the most unsuited for capturing coupling between highly coupled classes, even if there is no statistically significant difference between *structural*, *dynamic*, and *logical* coupling. The *dynamic* and the *semantic* coupling are the only ones that properly capture low coupling, having median=1 for both original and external developers (see Fig. 2). On the other hand, the *structural* measure seems to be completely out of sync with developers' perception of low coupling. This is also confirmed by the original developers' feedback, which perceived the pair of classes (ZoomTool, ZoomUpdateStrategy) as coupled (median=3) and one of them explained that "*these classes are peer features participating in the same context*". Also in this case, the *semantic* measure is the only one capturing some form of coupling between these two classes (i.e., CCBC=0.52). Nevertheless, we need to point out that the *semantic* measure does not represent a silver bullet. We also observed cases like the one for the class pair (AWTCursor, Locator) classified as not coupled by the *logical* measure, and confirmed as such by developers (median=1), where the *semantic* measure reports a coupling of 0.41 (not very high, nor as low as perceived by developers).

JHotDraw findings. The results confirm our previous findings obtained for ArgoUML: the *semantic* measure is the one that better approximates the coupling perceived by developers on pairs of highly coupled classes, while there is no big difference in performances of the other three measures. The *structural* measure is the least suited for approximating the perceived coupling for low coupled classes.

Finally, for **jEdit** most of the results observed on the other two systems are confirmed (see Figure 3). In particular, the *semantic* measure is still the one that better aligns with developers' perceptions for highly-coupled classes (median=4.5 for original developers and 4 for external), achieving also statistically significant differences when compared to the other three measures (p-value always < 0.01, medium/high effect size). The *logical* coupling is the second best measure for this system, with statistically significant differences with respect to the *structural* one (p-value < 0.01, with a medium effect size), which turned out to be the most inefficient for capturing high coupling. On this system, the *semantic* measure also works particularly well for capturing low coupling, as its performances are significantly better than the other measures (p-value < 0.01 in all cases, with a medium effect size).

jEdit findings. The *semantic* measure is, by far, the most aligned with developers' perception of coupling. The *structural* measure, as observed on JHotDraw, does not properly

capture cases of highly perceived coupling, while the *logical* coupling achieves good results.

Coupling captured by the experimented measures and perceived by developers. The *semantic* measure is the closest measure to the developers' perception of coupling. This result can be explained by the fact that developers embed their knowledge and design rationale in the comments and identifiers in code, information that is captured by the *semantic* measure. Surprisingly, the *structural* coupling is the one exhibiting more inconsistencies, especially when capturing low coupling, even more than the *semantic* measure.

C. To what extent do coupling measures reflect the mental model of developers when grouping together classes?

Table IV reports the descriptive statistics for the MoJoFM achieved by each of the four coupling measures when reconstructing the original software modularization of the three software systems. The *semantic* measure is the one achieving better reconstruction accuracy with respect to the original design (average MoJoFM=0.60), followed by *structural* (0.53), *dynamic* (0.43), and *logical* (0.04) measures. These results confirm what we observed in **RQ₂**: the *semantic* measure is the best one in capturing the developers' perceived coupling even if, overall, none of the measures achieve very good results in this evaluation. In addition, they suggest that semantic coupling should be considered as a useful source of information when performing modularization, although the research performed in the past mainly used static [18] or dynamic [39] information. It is also important to highlight the relatively low performance achieved by the *logical* measure, which may be not due to the low quality of coupling relationships, but rather to the incomplete information (i.e., coupling links) provided by this measure to the clustering algorithm. For example, for the 1,782,272 possible pairs of classes in ArgoUML, the *logical* coupling only provides information for about 180 pairs, making it impossible for the clustering algorithm to even approximate the original decomposition. In essence, this indicates that *logical* coupling could be used for some purpose (e.g., to identify change impacts [24], [32]), but may not be suitable for modularization purposes.

Ability to reconstruct the original modularization. The *semantic* measure is confirmed as the most suitable for capturing the coupling perceived by developers—in terms of the original modularization—followed by the *structural* and the *dynamic* measures.

V. THREATS TO VALIDITY

Threats to *construct validity* concern the relationship between theory and observation. They are primarily related to the way we measured *structural*, *dynamic*, *logical* and *semantic* coupling. For *structural* [21] and *conceptual* [8] coupling, we used measures that are available in literature. For *dynamic* coupling, we used a measure derived by the one defined by Arisholm *et al.* [15] that we adapted to capture the coupling